

Taller: Búsqueda en Grafos y Resolución de Laberintos

Por: Guillermo Andres De Mendoza Corrales

Tema: Grafos y Algoritmos de Búsqueda (DFS, BFS, Dijkstra's, A*, Greedy)

Fecha de entrega: Septiembre, Lunes 14 de 2026

1. Introducción

El objetivo de este taller es aplicar los conceptos de teoría de grafos para resolver un problema clásico de inteligencia artificial y estructuras de datos: la salida de un laberinto. Los estudiantes deberán utilizar una base de código proporcionada (que implementa la estructura del laberinto) y desarrollar la lógica de búsqueda.

El código de generación del algoritmo lo podrán encontrar en el archivo: `Mazze_generator.ipynb`

2. Objetivos

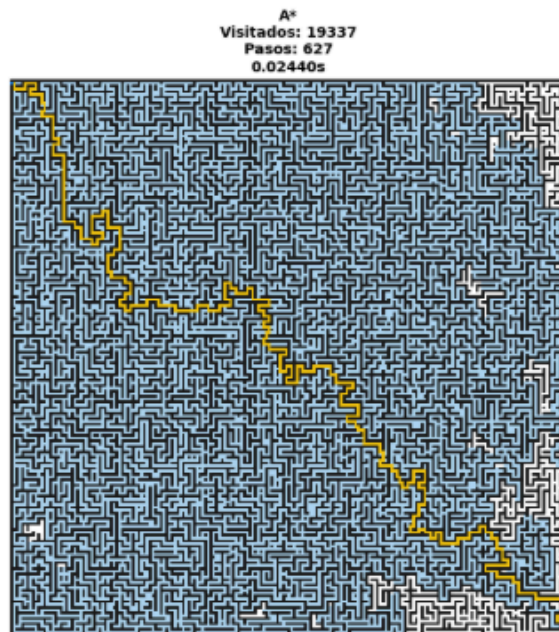
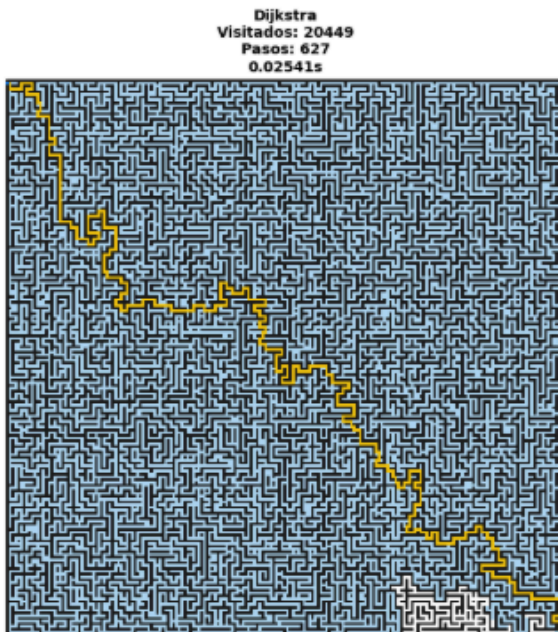
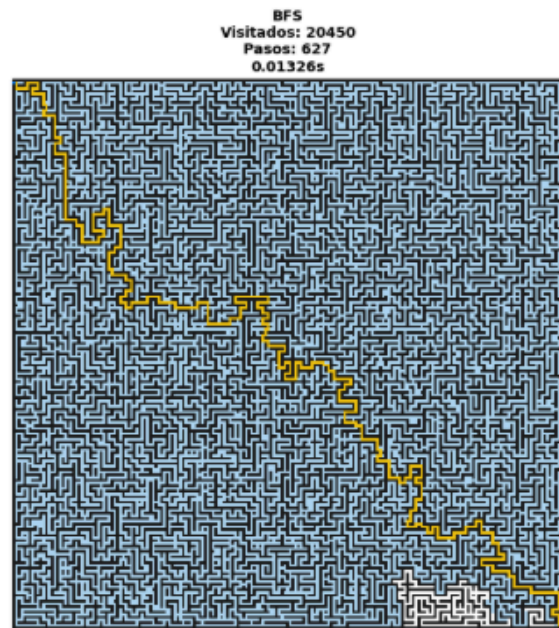
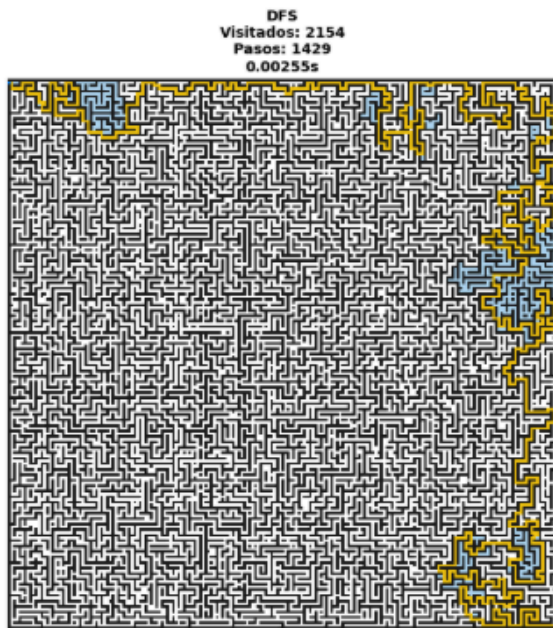
- Implementar los algoritmos de búsqueda: **DFS** (Búsqueda en Profundidad), **BFS** (Búsqueda en Anchura), **Dijkstra's**, **A*** (Búsqueda Informada) y Greedy Search..
- Medir y comparar el rendimiento de cada algoritmo bajo métricas objetivas.
- Visualizar el comportamiento de la exploración y la ruta final encontrada.

3. Requerimientos de Implementación

Para cada uno de los tres algoritmos, el programa debe ser capaz de calcular y registrar:

1. **Tiempo de ejecución:** Tiempo total empleado por el algoritmo para encontrar la solución (en milisegundos o microsegundos).
2. **Nodos Visitados:** Cantidad total de nodos explorados por el algoritmo antes de llegar a la meta. Se debe generar una gráfica o representación visual que muestre qué celdas fueron exploradas.
3. **Camino Solución:** La secuencia de nodos que conectan el inicio con el fin. Se debe generar una gráfica que resalte este camino específico sobre el laberinto.

ejemplo:



4. Algoritmos a Implementar

A. Búsqueda en Profundidad (DFS)

- **Mecánica:** Utilizar una Pila (Stack).
- **Control de ciclos:** Es obligatorio marcar los nodos visitados.
- **Propiedad:** Observar si el camino encontrado es el más corto o simplemente el primero que encontró.

B. Búsqueda en Anchura (BFS)

- **Mecánica:** Utilizar una Cola (Queue).

- **Propiedad:** Verificar matemáticamente que el camino obtenido sea el camino más corto en un grafo sin pesos.

C. Dijkstra's

- **Mecánica:** Utilizar una Lista Ordenada.
- **Conteo:** Implementar el conteo de pasos "steps" para cada ruta seleccionada.

D. Algoritmo A* (A-Star)

- **Mecánica:** Utilizar una Lista Ordenada.
- **Heurística:** Implementar la Distancia Manhattan como estimación del costo restante.

E. Greedy Search:

- **Mecánica:** Utilizar una Lista Ordenada.
- **Heurística:** Implementar la Distancia Manhattan como estimación del costo restante.

5. Análisis y Comparación

Al finalizar las implementaciones, el estudiante debe completar la siguiente tabla comparativa:

Métrica	DFS	BFS	Dijkstra's	A*	Greedy
Tiempo de Ejecución					
Total Nodos Visitados					
Longitud del Camino Solución					

Es deseable si el estudiante evalúa diferentes parámetros en el algoritmo, como los tamaños del tablero o más huecos en las paredes.

6. Entregables

- Código fuente debidamente comentado.
- Gráficas y valores generados por el programa para cada algoritmo mostrando:
 - Los nodos visitados (frente de exploración).
 - El camino solución final.
 - Tiempo de ejecución
- Informe con la tabla comparativa y respuestas a las preguntas de reflexión.